

LAB 02

System Performance Evaluation Using Benchmarking: Sieve of Eratosthenes

System Configuration and Performance Evaluation Lab
Course No: 0613-310

Ekramul Haque Amin
Lecturer, AI and Data Science
Dhaka International University (DIU)

From Lab 01 to Lab 02

Lab 01 showed

Under heavy load, the same benchmark took longer to finish.

Today we ask

Did the program do more work, or did it just wait longer?

To answer this, we measure two different times: CPU time and wall time

Today's Objectives

1. Understand what a benchmark is and its main types
2. Implement the Sieve of Eratosthenes as a benchmark program
3. Measure CPU time and wall time under Normal and Loaded conditions
4. Use mean, standard deviation and CV to judge consistency

Deliverable: Lab Report 02

What Is a Benchmark?

A **fixed program** that is run on different systems or conditions so their performance can be **compared**.

- Same program and same input every time
- Same way of measuring time
- Only the system or its condition changes

If anything else changes, the comparison is not fair

Types of Benchmark Workloads

Type	What it is	Examples
Instruction mix	Weighted list of instructions and how often each is used	Gibson mix
Kernel ← today	The small, most time-consuming core of a real program	Sieve, matrix inversion
Synthetic program	Artificial program built to imitate a real workload	Whetstone, Dhrystone
Application benchmark	A real application with a real workload	SPEC, Debit-Credit

The Sieve is a kernel: small, but it stresses the CPU and memory

Why Use the Sieve as a Benchmark?

CPU-bound

Millions of loop steps, with no waiting for disk or network

Uses memory

Reads and writes a large array of $n + 1$ values

Same answer every time

664,579 primes below 10,000,000, so the result is easy to check

Simple to write

Easy to code in any language, so systems can be compared fairly

How the Sieve Works: $n = 30$

2	3	4	5	6	7	8	9	10	11
12	13	14	15	16	17	18	19	20	21
22	23	24	25	26	27	28	29	30	

Cross out multiples of 2, then 3, then 5. Stop at $\sqrt{30} \approx 5.5$

Blue boxes = 10 primes kept · Crossed-out grey boxes = composite

The Algorithm, Step by Step

1. Make a list `is_prime[0 ... n]`, all True; set 0 and 1 to False
2. For each i from 2 to $\sqrt{n}$: if i is still True, mark i^2 , $i^2 + i$, $i^2 + 2i$, ... as False
3. Start at i^2 , because smaller multiples were already crossed out by smaller primes
4. Count the values that are still True

Time complexity: $O(n \log \log n)$, almost linear in n

CPU Time vs Wall Time

CPU Time

- Time the CPU actually spent running our program
- Does not count waiting
- `time.process_time()`

Wall Time

- Real clock time from start to finish
- Counts waiting for the CPU
- `time.perf_counter()`

Wall Time = CPU Time + Waiting Time

Where Does the Time Go?

Normal



CPU working

Wall time $\approx$ CPU time

Loaded



CPU

wait

CPU

wait

CPU

wait

CPU

Wall time = CPU time + waiting time

Same amount of CPU work in both rows. Under load, the OS gives the CPU to other processes in turns.

What Should We Expect?

Metric	Normal	Loaded	Reason
CPU Time	Baseline	About the same	Same work, same instructions
Wall Time	$\approx$ CPU time	Much larger	Program waits for its turn
Waiting (Wall – CPU)	Close to 0	Large	Other processes use the CPU

Our test run (N = 10,000,000):

Normal: CPU 0.86 s, Wall 0.86 s · Loaded: CPU 0.87 s, Wall 2.18 s

Why Run It 5 Times?

Every run is a little different, because of background tasks, caching and CPU speed changes. We summarize 5 runs with:

Mean

$$\bar{x} = \Sigma x / n$$

The typical time

Std. Deviation

$$s = \sqrt{(\Sigma (x - \bar{x})^2 / (n - 1))}$$

How far runs spread from the mean

CV

$$CV = s / \bar{x} \times 100\%$$

Spread as a % of the mean

Smaller CV = more consistent results

Code: The Sieve Function

```
def sieve(n):  
    is_prime = [True] * (n + 1)  
    is_prime[0] = is_prime[1] = False  
  
    for i in range(2, int(math.sqrt(n)) + 1):  
        if is_prime[i]:  
            for m in range(i * i, n + 1, i):  
                is_prime[m] = False  
  
    return sum(is_prime)    # counts the True values
```

Note the + 1 in the loop: without it, 49 would be counted as prime

Code: Measuring Both Times

```
def run_once(n):  
    wall_start = time.perf_counter()    # real clock time  
    cpu_start = time.process_time()     # CPU time of this process  
  
    count = sieve(n)  
  
    cpu_time = time.process_time() - cpu_start  
    wall_time = time.perf_counter() - wall_start  
    return count, cpu_time, wall_time
```

Start both clocks just before the work, and stop both just after it

Code: Repeat 5 Times and Summarize

```
cpu_times, wall_times = [], []
for run in range(1, RUNS + 1):          # RUNS = 5
    count, cpu_t, wall_t = run_once(N)   # N = 10,000,000
    cpu_times.append(cpu_t)
    wall_times.append(wall_t)

for name, values in (("CPU Time", cpu_times),
                     ("Wall Time", wall_times)):
    avg = statistics.mean(values)
    sd = statistics.stdev(values)
    cv = sd / avg * 100
```

Sample Output: Normal Condition

N = 10,000,000 Runs = 5

```
-----  
Run      Primes    CPU Time (s)    Wall Time (s)  
-----  
1         664,579        0.9831         0.9851  
2         664,579        0.8015         0.8018  
3         664,579        0.8352         0.8354  
4         664,579        0.9032         0.9036  
5         664,579        0.7631         0.7633  
-----  
CPU Time   Average = 0.8572 s   SD = 0.0872 s   CV = 10.17 %  
Wall Time  Average = 0.8579 s   SD = 0.0879 s   CV = 10.24 %  
Average waiting time (Wall - CPU) = 0.0006 s
```

Check: 664,579 primes in every run. If not, the program is wrong.

Task 1: Normal System Condition

1. Close all unnecessary applications
2. Check Task Manager: CPU usage should be low
3. Run: `python sieve_benchmark.py`
4. Do not touch the computer until it finishes
5. Screenshot the output; record the 5 runs in Table 1

Only the benchmark program should be running

Task 2: Loaded System Condition

1. Create heavy load: browser tabs with videos, compiling, other heavy apps
2. Best method: open several terminals and in each run:

```
python -c "while True: pass"
```

1. Use one more terminal than the number of logical cores
2. Run the same script again; record the 5 runs in Table 2
3. Press Ctrl + C in every terminal when finished

Recording the Results

Table 1 (Normal) and Table 2 (Loaded)

Run No.	CPU Time (s)	Wall Time (s)
1		
2		
3		
4		
5		
Average		

Table 3: Comparison

Metric	Normal	Loaded
Avg. CPU Time (s)		
Avg. Wall Time (s)		
Avg. Waiting (s)		
CV of Wall Time (%)		

Waiting = Avg. Wall – Avg. CPU

Analysis Questions

1. Compare performance under Normal and Loaded conditions
2. Which operating condition produced better performance?
3. How did increased system load affect the CPU time?
4. How did increased system load affect the wall time?
5. Explain the reasons behind the observed differences
6. Were the results consistent across all runs? Use the CV

Answer with your own numbers, not general statements

Common Mistakes to Avoid

Mistake	Fix
n is too small (e.g. 100)	Use n large enough that one run takes ≥ 1 second
Calling perf_counter() “CPU time”	CPU time = process_time()
Loop to $\text{int}(\sqrt{n})$ without + 1	<code>range(2, $\text{int}(\sqrt{n}) + 1$)</code>
Timing the print statements	Time only the sieve call
Changing code between conditions	Same script, same n, for both runs

Key Takeaway

Load does not make the program do more work. It makes the program **wait longer.**

CPU time stays about the same · Wall time grows

Next lab: Synthetic benchmarks: Ackermann's function and matrix multiplication